บทที่ 9 กราฟ (Graph)

1. กราฟคืออะไร

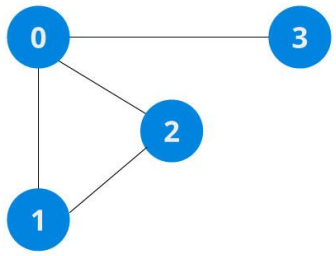
- กราฟ G เป็นโครงสร้างข้อมูลที่ประกอบด้วยเซตจำกัดของโหนดหรือจุด (Nodes/Vertices : V) และเซตของเส้น (Edges : E) ที่เชื่อมจุดเข้าด้วยกัน
- โดยแต่ละเส้นแทนด้วยคู่ลำดับของจุด เช่น (a, b) หมายถึงเส้นที่เชื่อมจุด a และ b ซึ่งแต่ละคู่ของจุดจะมีเส้นเชื่อมหรือไม่มีเส้นเชื่อมก็ได้ ถ้ามีเส้นเชื่อมสามารถมีได้เส้นเดียว หรือมากกว่า 1 เส้นก็ได้

<u>ตัวอย่าง1</u>

V = {0, 1, 2, 3}

E = {(0,1), (0,2), (0,3), (1,2)}

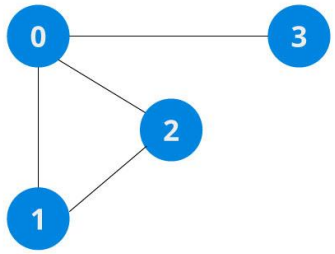
G = {V,E}



- จากภาพข้างต้น กราฟนี้ประกอบด้วย 4 จุด ได้แก่ 0, 1, 2 และ 3
- กราฟที่ไม่มีจุดเรียกว่า กราฟว่าง (Empty graph)
- ดีกรีของจุด (degree) หมายถึง จำนวนเส้นที่เชื่อมจุดนั้น จากภาพข้างต้น

จุด	จำนวนดีกรี
0	3
1	2
2	2
3	1

- จุดใกล้เคียง/เชื่อมโยง/ประชิด (adjacent vertices) หมายถึงจุดที่มีเส้นเชื่อมถึงกัน เช่น จากภาพจุดใกล้เคียงจุด 0 คือ 1,2 และ 3 ในขณะที่จุดใกล้เคียงจุด 2 คือ 0 และ1 เป็นต้น



- เส้นทาง (path) คือ กลุ่มของจุดที่แสดงเส้นทางจากจุด V_i ถึงจุด V_k โดยที่จุด V_i และ V_{i+1} เป็นจุดใกล้เคียงกัน
    - ความยาวของเส้นทางจะมีค่าเท่ากับจำนวนเส้นในเส้นทางนั้น จากภาพข้างต้น กำหนดเส้นทางจากจุด 0 ถึง 1 คือ π : 0, 2, 1 ดังนั้นเส้นทางนี้มีความยาวเป็น 2
    - เส้นทางที่ไม่มีจุดซ้ำกันเลย เรียกว่า เส้นทางแบบง่าย (simple path) ดังนั้น 0, 2, 1 เป็นเส้นทางแบบง่าย ในขณะที่ 0, 1, 2, 0, 3 ไม่ใช่เส้นทางแบบง่ายเพราะว่ามีทางเดินไปยังจุด 0 ซ้ำ

2. ประเภทกราฟ

กราฟสามารถแบ่งได้เป็นหลายประเภทตามลักษณะดังนี้

2.1 แบ่งตามทิศทาง ได้แก่ กราฟไม่ระบุทิศทาง และกราฟระบุทิศทาง

2.2 แบ่งตามการมีวงจรปิด ได้แก่ Cyclic graph และ Acyclic graph

2.3 แบ่งตามการมี parallel และ loop ได้แก่ Simple graph, Multi graph และ Pseudo graph

2.4 จำนวนดีกรีของแต่ละโหนดในกราฟ ได้แก่ Discrete graph, Linear graph, Regular graph และ Complete graph

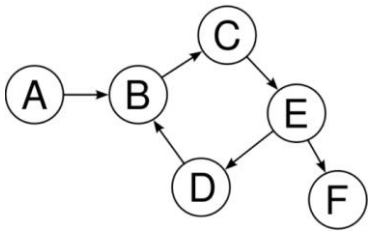
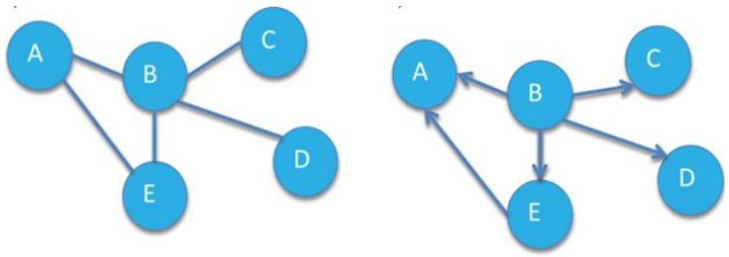
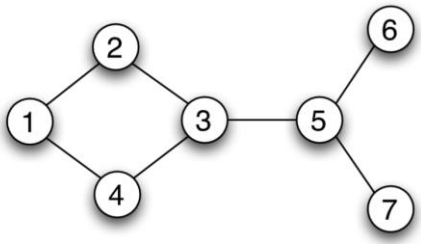
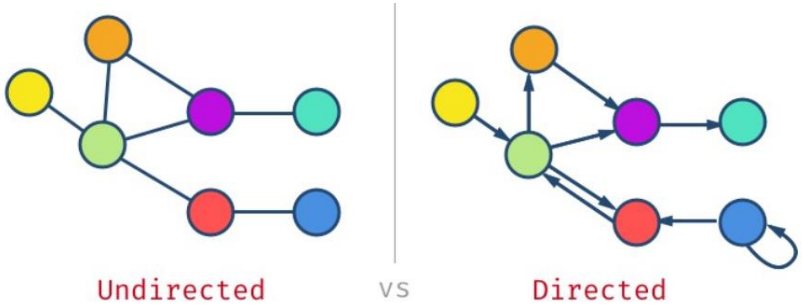
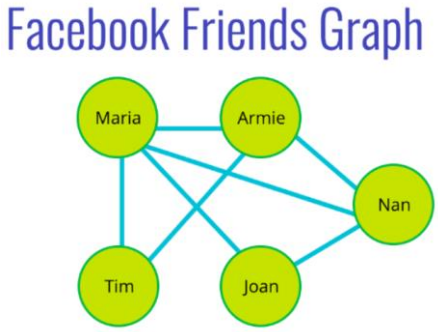
2.5 กราฟมีน้ำหนัก (weighted graph)

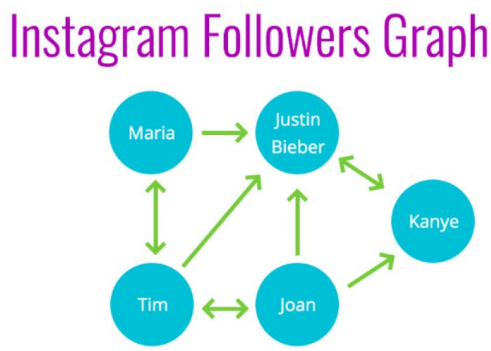
แต่ละประเภทของกราฟ มีรายละเอียดดังนี้

2.1 แบ่งตามทิศทาง ได้แก่ กราฟไม่ระบุทิศทาง และกราฟระบุทิศทาง

2.1.1 **กราฟไม่ระบุทิศทาง** (Undirected graph) ทุกเส้นในกราฟไม่มีการระบุทิศทางการเชื่อมต่อระหว่างจุด

2.1.2 **กราฟระบุทิศทาง** (Directed graph) ทุกเส้นในกราฟมีการระบุทิศทางการเชื่อมระหว่างจุด

	
	
	A real-world example of this is when you add a friend on Facebook. Each user now has full access to the other user's public content.

	A real-world example of a directed graph is followers on Instagram. When you follow a new account, that new account does not automatically follow you back.

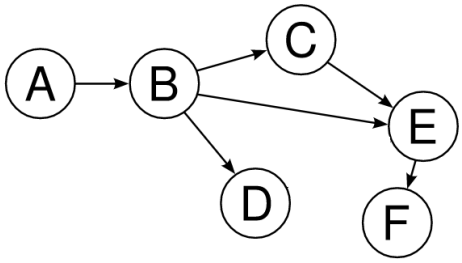
ดีกรีสำหรับกราฟที่มีทิศทาง จะแบ่งได้เป็น In-degree (Indeg) and 0ut-degree (Outdeg)

- In-degree จำนวนเส้นที่ชี้เข้าที่โหนด
- Out-degree จำนวนเส้นที่ชี้ออกจากโหนด

ซึ่งดีกรีของกราฟ หาได้จาก Degree = Indeg + Outdeg

<u>ตัวอย่าง2</u>

Node	Indeg	Outdeg
A	0	1
B	1	3
C	1	1
D	1	0
E	2	1
F	1	0

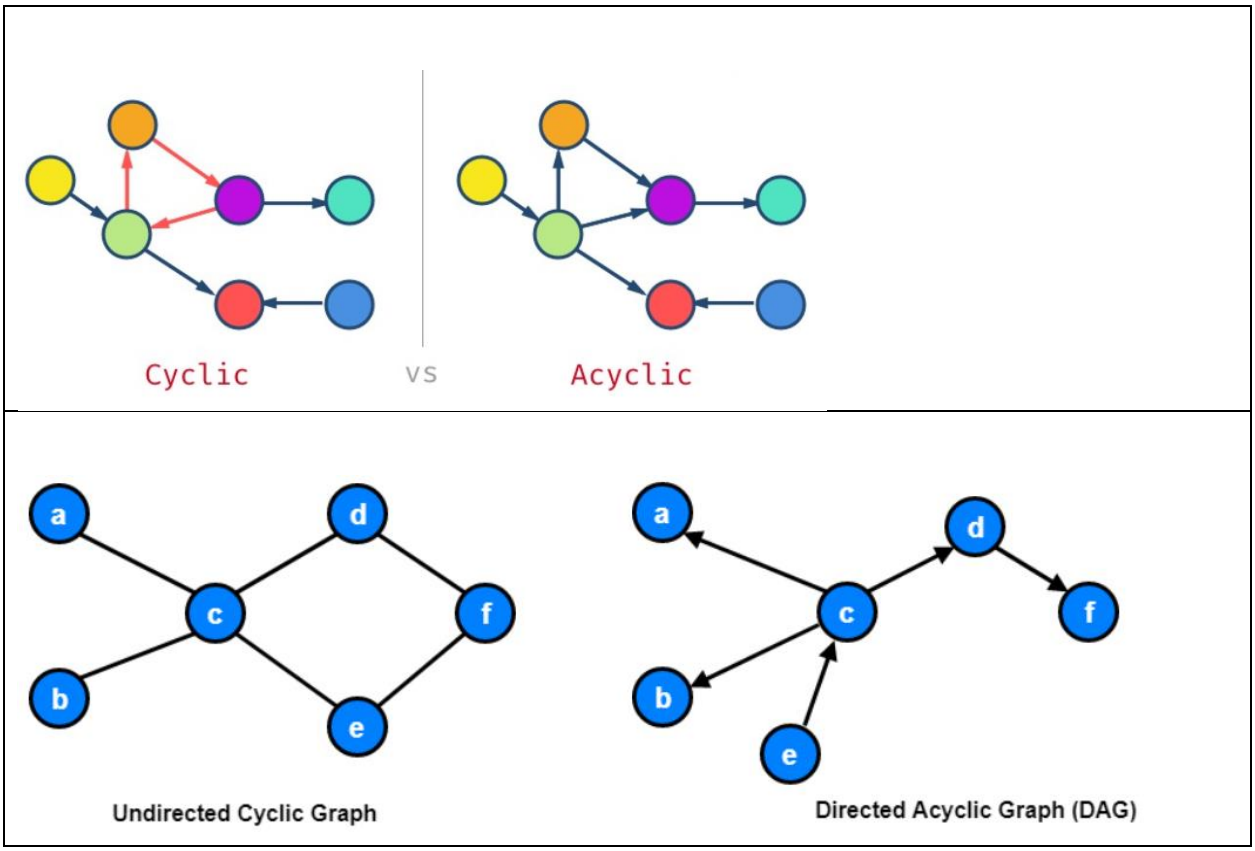


ดีกรีของกราฟ เท่ากับ .....

2.2 แบ่งตามการมีวัฏจักร ได้แก่ Cyclic graph และ Acyclic graph

2.2.1 **Cyclic graph** หมายถึงกราฟที่มีส่วนในส่วนหนึ่งของประกอบด้วย path ที่มีจุดเริ่มต้นและจุดปลายทางเป็นจุดเดียวกัน (a cyclic graph is a graph containing at least one graph cycle.)

2.2.2 **Acyclic graph** is a graph having no graph cycles.

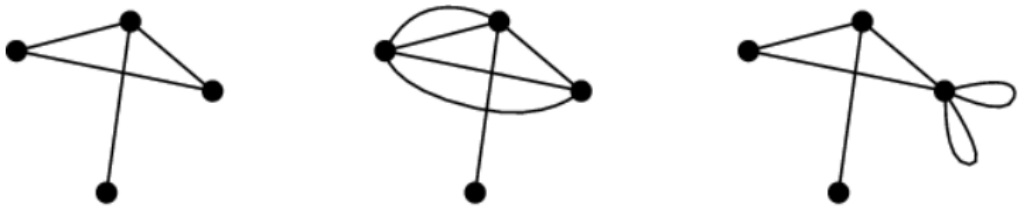


2.3 แบ่งตามการมี parallel และ loop ได้แก่ Simple graph, Multi graph และ Pseudo graph

2.3.1 **กราฟทั่วไป (Simple graph)** คือกราฟที่แต่ละคู่ของจุด มีเส้นเชื่อมเพียงเส้นเดียว

2.3.2 **มัลติกราฟ (Multi graph)** คือกราฟที่บางคู่ของจุดสามารถมีเส้นเชื่อมได้มากกว่าหนึ่งเส้น (parallel edges) แต่ไม่มีลูป (loop/self-loop)

2.3.3 **ซูโดกราฟ (Pseudo graph)** คือกราฟที่บางคู่ของจุดสามารถมีเส้นเชื่อมได้มากกว่าหนึ่งเส้น และมีลูป

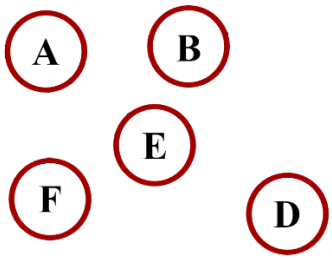
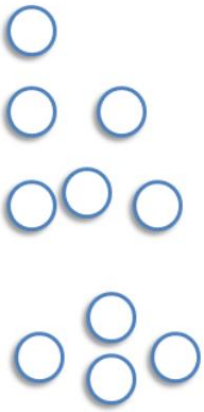


Simple graph Multi graph Pseudo graph

2.4 แบ่งตามจำนวนดีกรีของแต่ละโหนดในกราฟ เมื่อให้ n = จำนวนจุดในกราฟ

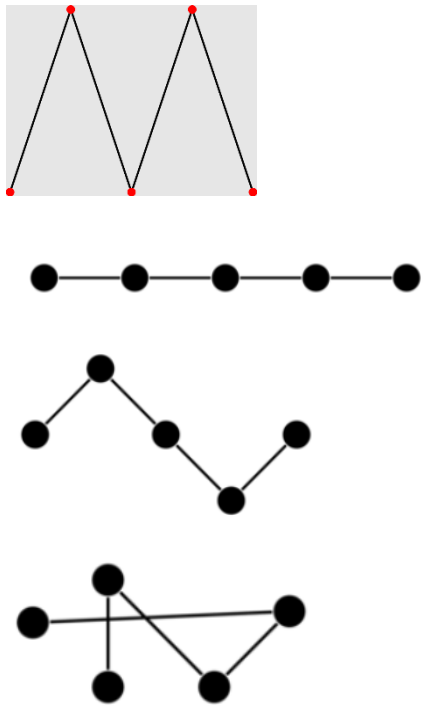
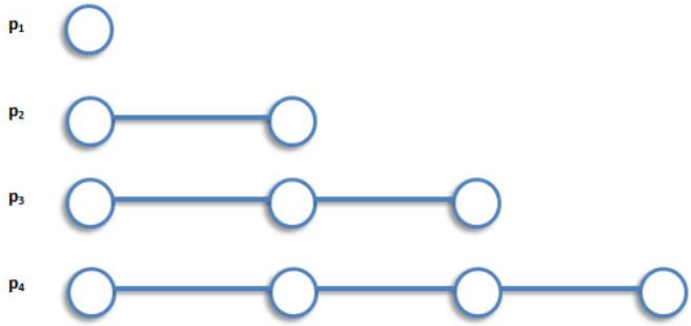
2.4.1 **Discrete graph (Null graph)** หรือกราฟจุด หมายถึงกราฟที่มีลักษณะเป็นจุด แต่ไม่มีเส้นเชื่อมระหว่างจุด (a graph contains no edge) ดังนั้นกราฟจุดจึงเป็น Disconnected graph

<u>ตัวอย่าง3</u> กราฟจุด

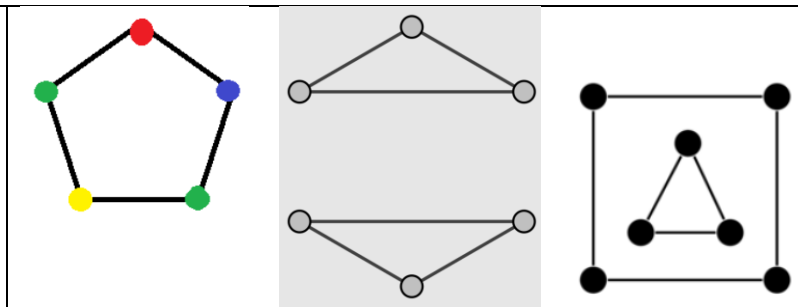
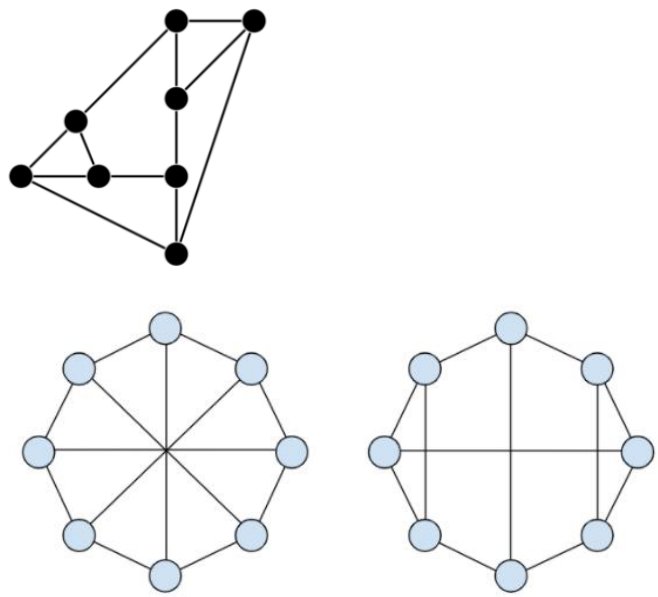
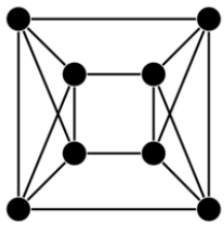
n = 5	
n = 1, 2, 3, 4	

2.4.2 **Linear graph หรือ Path graph** คือกราฟที่มี n $\geq$ 2 โดยแต่ละคู่ของจุดจะมีเส้นเชื่อมถึงกันเป็นลำดับ (a graph whose vertices can be listed in the order v_1, v_2, ..., v_n such that the edges are $\{v_i, v_{i+1}\}$ where i = 1, 2, ..., n – 1) โดยที่ดีกรีของทุกจุดเป็น 2 ยกเว้นจุดแรกและจุดสุดท้ายมีดีกรีเป็น 1

<u>ตัวอย่าง4</u> Linear graph

n = 5	
n = 1, 2, 3, 4	

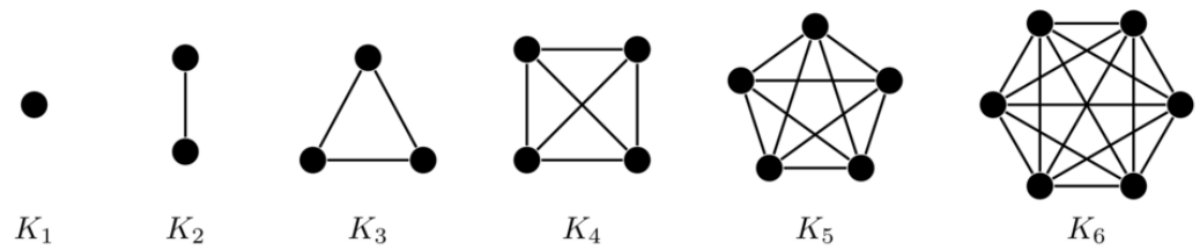
2.4.3 Regular graph: a graph is called regular graph if degree of each vertex is equal. A graph is called **K-Regular** if degree of each vertex in the graph is K.

2-Regular	
3-Regular	
What-Regular?	

คำถาม

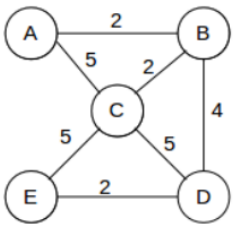
1. จงวาด 2-Regular ที่ n = 8
2. จงวาด 4-Regualr ที่ n = 5

2.4.4 Complete graph is a simple graph in which every vertex is adjacent to every other vertex.

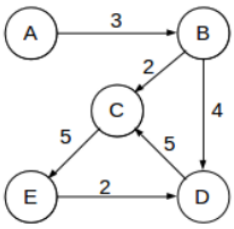


2.5 กราฟมีน้ำหนัก หรือกราฟถ่วงน้ำหนัก (weighted graph)

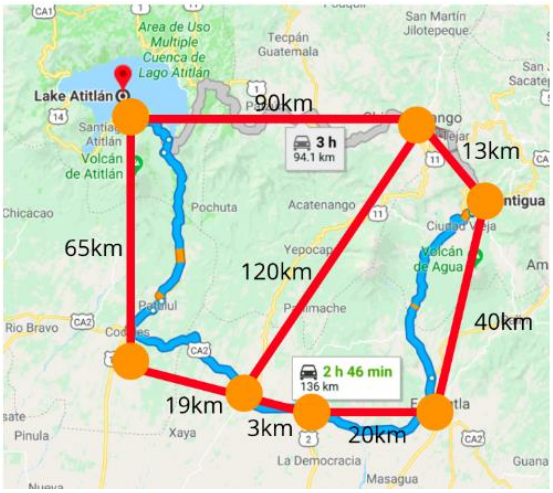
- กราฟที่แต่ละเส้นในกราฟมีการให้น้ำหนัก ค่าน้ำหนักต้องไม่เป็นค่าลบ
- น้ำหนักอาจเป็นระยะทาง ความเร็ว เวลาในการเดินทาง ค่าธรรมเนียม ค่าใช้จ่ายในการก่อสร้าง หรือค่าโดยสาร เป็นต้น

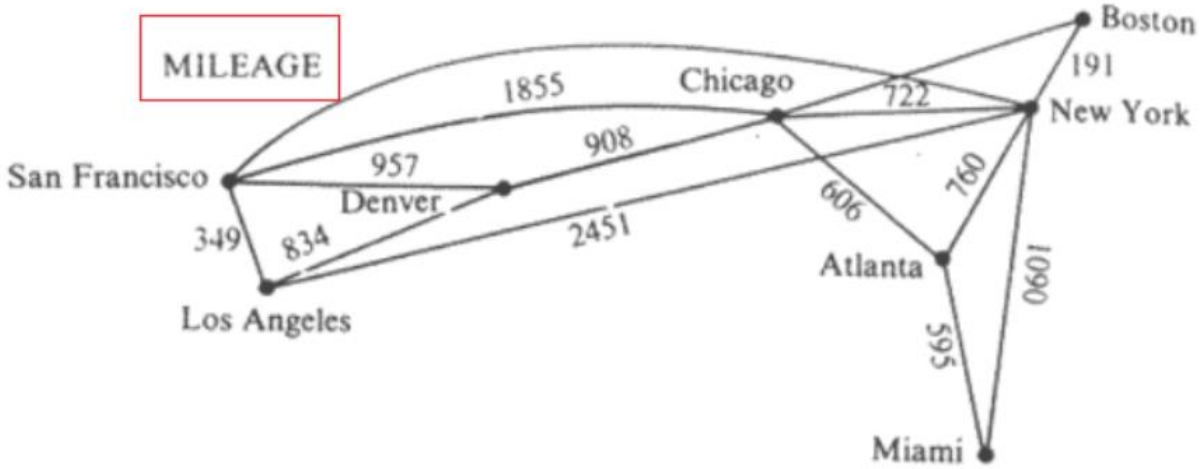


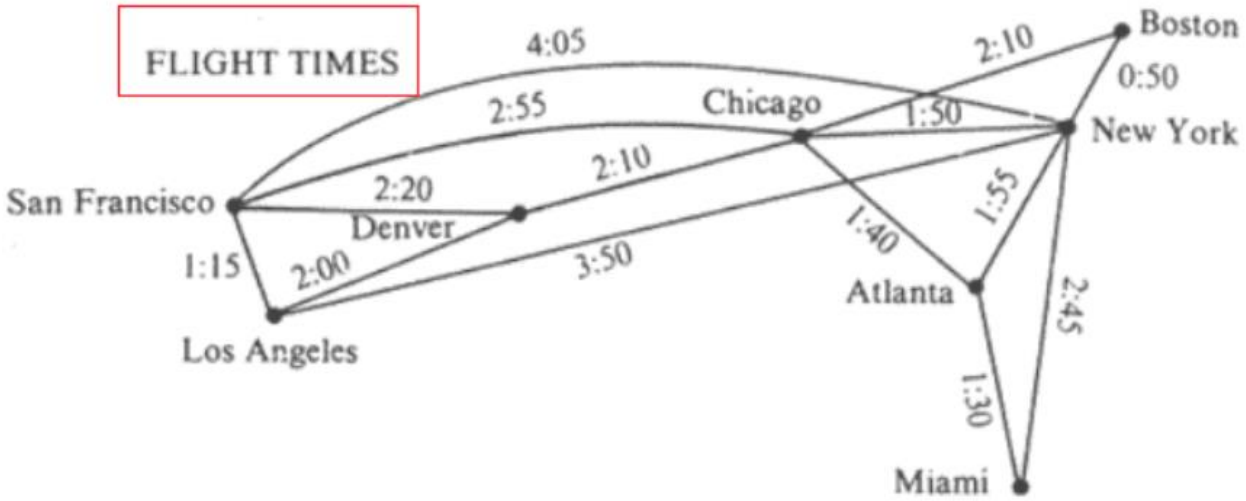
Undirected



Directed



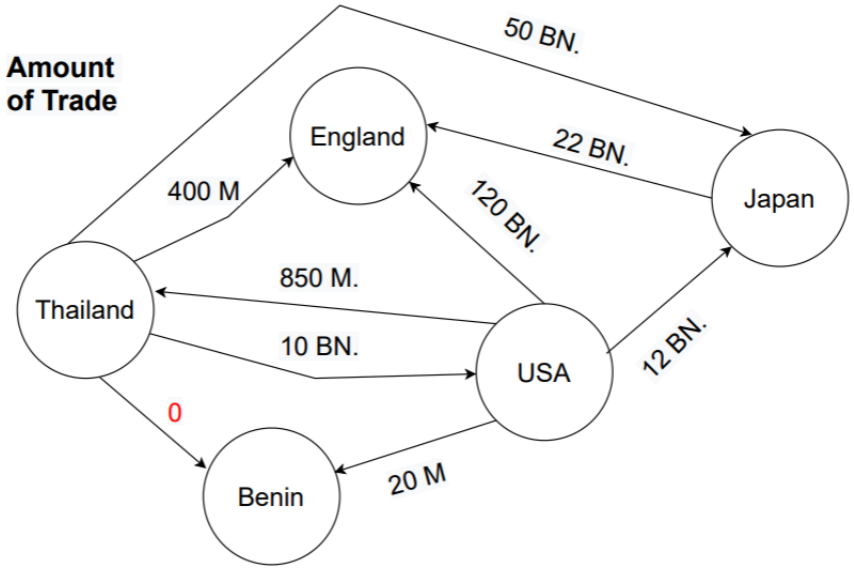




กราฟถ่วงน้ำหนักมีค่าน้ำหนักเป็นศูนย์ได้หรือไม่

- If your graph represents countries and the *amount of trade* done between them, a zero-weight edge would mean no trade, which is the same as having no edge at all.
- If your graph represents *distances*, a zero-weight edge would correspond to two places at distance zero from each other, which would mean they'd actually be the same place, so should both be represented by the same vertex.
- If your graph represents a *road network* and edge weights represent the amount of traffic, there's a big difference between a road that nobody uses (zero-weight edge) and no road at all (no edge). In this case, weight can be zero.

<u>ตัวอย่าง</u> Weight = 0



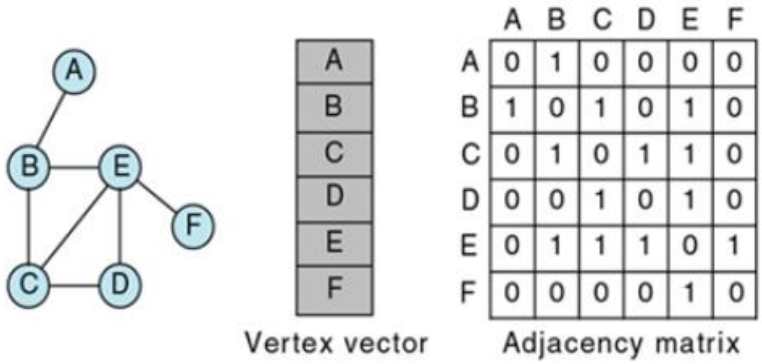
2.6 การนำเสนอกราฟ (Adjacency Matrix and Adjacency List)

ทำได้ 2 วิธีได้แก่ Adjacency Matrix และ Adjacency List ดังรายละเอียดต่อไปนี้

2.6.1 Adjacency Matrix คือ การเก็บข้อมูลหรือแจกแจงความสัมพันธ์ภายในกราฟที่ใช้โครงสร้างของเมทริกซ์ (Matrix) โดยใช้เมทริกซ์ขนาด 2 มิติเพื่อแสดงการเชื่อมโยงจุดต่าง ๆ ในกราฟ

- ถ้าจุดในกราฟเชื่อมต่อกันมีค่าเป็น 1
- ถ้าจุดในกราฟไม่เชื่อมต่อกันมีค่าเป็น 0

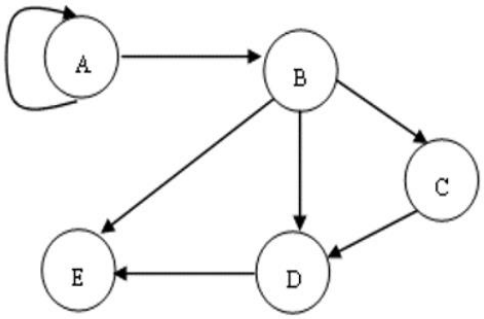
<u>ตัวอย่าง</u>



	A	B	C	D	E	F
A	0	1	0	0	0	0
B	1	0	1	0	1	0
C	0	1	0	1	1	0
D	0	0	1	0	1	0
E	0	1	1	1	0	1
F	0	0	0	0	1	0

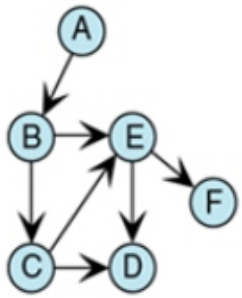
(a) Adjacency matrix for non-directed graph

<u>ตัวอย่าง</u>



	A	B	C	D	E
A	1	1	0	0	0
B	0	0	1	1	1
C	0	0	0	1	0
D	0	0	0	0	1
E	0	0	0	0	0

<u>ตัวอย่าง</u>



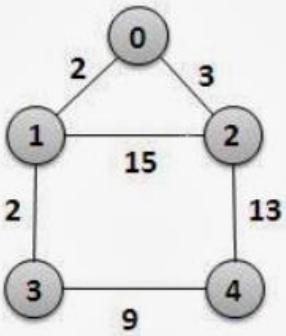
Vertex vector
A
B
C
D
E
F

Adjacency matrix

	A	B	C	D	E	F
A	0	1	0	0	0	0
B	0	0	1	0	1	0
C	0	0	0	1	1	0
D	0	0	0	0	0	0
E	0	0	0	1	0	1
F	0	0	0	0	0	0

(a) Adjacency matrix for directed graph

<u>ตัวอย่าง</u>



	0	1	2	3	4
0	0	2	3	0	0
1	2	0	15	2	0
2	3	15	0	0	13
3	0	2	0	0	9
4	0	0	13	9	0

Adjacency Matrix Representation of Weighted Graph

ข้อดี Adjacency Matrix:

1. ง่ายต่อการใช้งาน: การตรวจสอบว่าจุดใดเชื่อมกันหรือไม่สามารถทำได้อย่างรวดเร็วเพียงแค่เข้าถึงค่าในเมตริกซ์

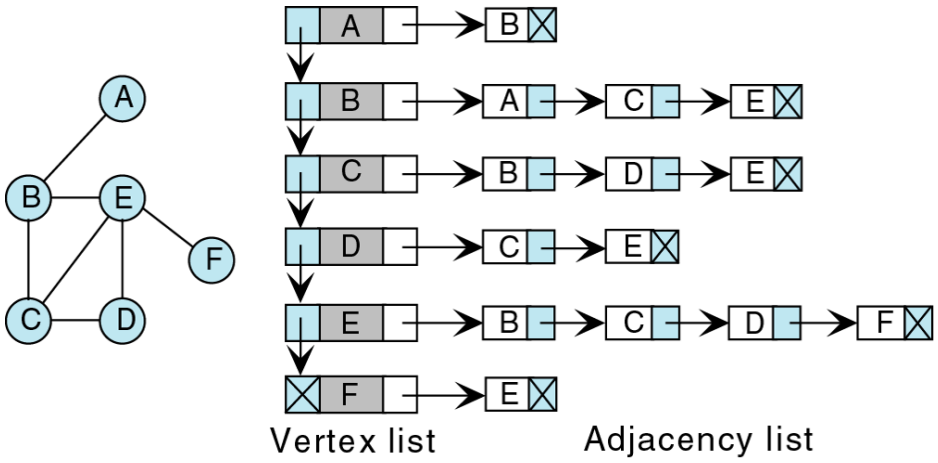
2. ประหยัดเวลาในการค้นหา: การค้นหาความสัมพันธ์ระหว่างจุดใช้เวลาเพียง O(1)

ข้อเสีย Adjacency Matrix:

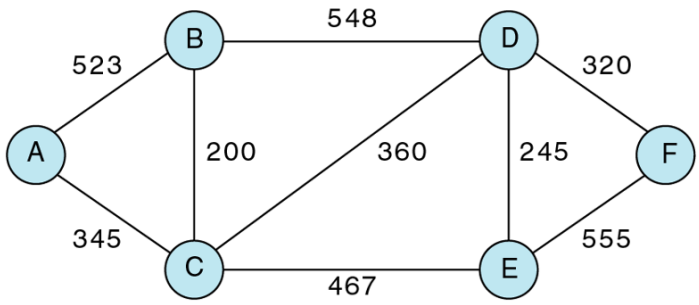
1. ใช้พื้นที่มาก: หากกราฟมีจุดมากและเชื่อมต่อเพียงไม่กี่คู่จะทำให้เมทริกซ์มี value มากที่เป็น 0 หรือที่เรียกว่ากราฟบาง (Sparse Graph) ซึ่งสิ้นเปลืองเนื้อที่
2. ไม่เหมาะกับกราฟขนาดใหญ่: การจัดเก็บกราฟขนาดใหญ่อาจต้องการหน่วยความจำมาก

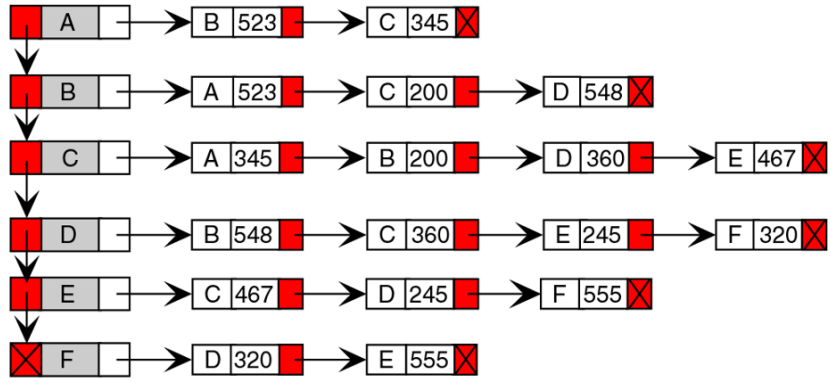
2.6.2 Adjacency List คือ การเก็บข้อมูลแบบลิงค์ลิสต์ โดยแต่ละโหนดจะจัดเก็บจุด และใช้ pointer ในการเชื่อมไปยังโหนดที่มีการเชื่อมถึงกัน ซึ่งโหนดแรกจะเป็นจุดเริ่มต้น โหนดอื่นที่อยู่ถัดไปในลิสต์จะเป็นจุดที่เชื่อมต่อกับจุดเริ่มต้นเท่านั้น

<u>ตัวอย่าง</u>

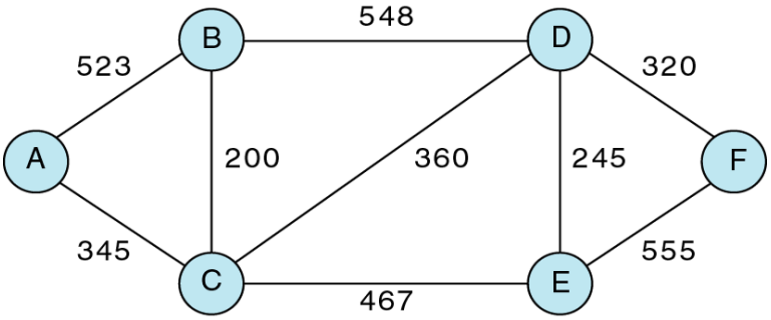


<u>ตัวอย่าง</u>





Vertex list Adjacency list



A
B
C
D
E
F

Vertex vector

	A	B	C	D	E	F
A	0	523	345	0	0	0
B	523	0	200	548	0	0
C	345	200	0	360	467	0
D	0	548	360	0	245	320
E	0	0	467	245	0	555
F	0	0	0	320	555	0

Adjacency matrix

3. การท่องเข้าไปในกราฟ (Graph Traversal)

การท่องเข้าไปในกราฟ คือ การเข้าถึงทุกจุดในกราฟ เพื่อทำการสำรวจ (explore) การท่องเข้าไปในกราฟทำได้ 2 วิธี ดังนี้

3.1 การท่องเข้าไปในแนวกว้าง (Breadth First Traversal/Search)

3.2 การท่องเข้าไปในแนวลึก (Depth First Traversal/Search)

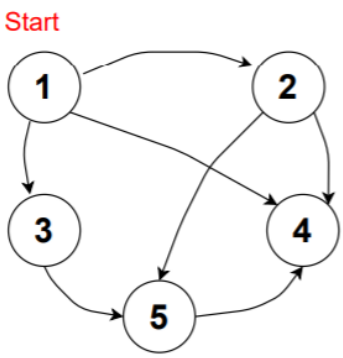
รายละเอียดแต่ละวิธี อธิบายได้ดังนี้

3.1 **การท่องเข้าไปในแนวกว้าง** (Breadth First Traversal/Search: BFS)

มีขั้นตอนการทำงานดังนี้

1. เลือกจุดในกราฟ ขึ้นมาเป็นจุดเริ่มต้น (visited)
2. นำจุดที่เชื่อมโยง/ประชิดกับจุดที่เลือก ไปเก็บไว้ใน<u>คิว (queue)</u>
3. เอาข้อมูลออกจากคิว (dequeue) และนำทุกจุดที่เชื่อมโยง/ประชิดกับจุดที่เอาออก (ถ้ามี) ไปเก็บไว้ในคิว (enqueue)
4. ทำซ้ำในขั้นตอนที่ 3 จนกระทั่งทุกจุดถูกแวะเยี่ยม

<u>ตัวอย่าง</u> BFS



visited	1				
queue	3	4	2		

visited	1				
queue	3	4	2		

dequeue

visited	1	3			
queue	4	2	5		

dequeue

visited	1	3	4		
queue	2	5			

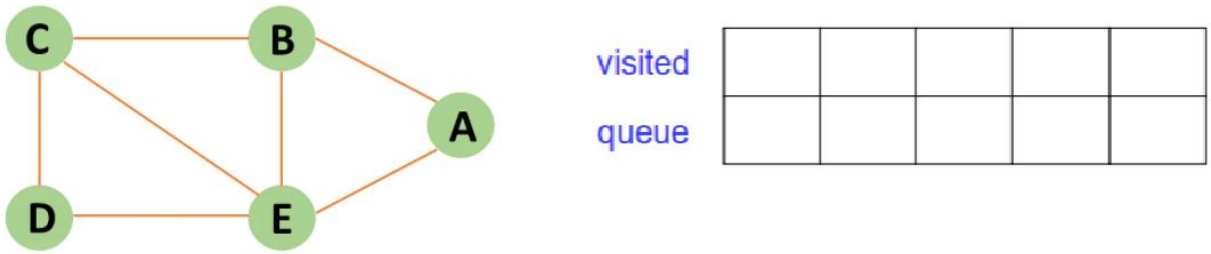
dequeue

visited	1	3	4	2	
queue	5				

dequeue

visited	1	3	4	2	5
queue					

<u>ตัวอย่าง</u> BFS



Python : BFS 1

```
# Python3 Program to print BFS traversal
# from a given source vertex. BFS(int s)
# traverses vertices reachable from s.
from collections import defaultdict

# This class represents a directed graph
# using adjacency list representation
class Graph:

    # Constructor
    def __init__(self):

        # default dictionary to store graph
        self.graph = defaultdict(list)

    # function to add an edge to graph
    def addEdge(self,u,v):
        self.graph[u].append(v)

    # Function to print a BFS of graph
    def BFS(self, s):

        # Mark all the vertices as not visited
        visited = [False] * (max(self.graph) + 1)

        # Create a queue for BFS
        queue = []

        # Mark the source node as
        # visited and enqueue it
```

```
        queue.append(s)
        visited[s] = True

        while queue:

            # Dequeue a vertex from
            # queue and print it
            s = queue.pop(0)
            print (s, end = " ")

            # Get all adjacent vertices of the
            # dequeued vertex s. If a adjacent
            # has not been visited, then mark it
            # visited and enqueue it
            for i in self.graph[s]:
                if visited[i] == False:
                    queue.append(i)
                    visited[i] = True

# Driver code

# Create a graph given in
# the above diagram
g = Graph()
g.addEdge(0, 1)
g.addEdge(0, 2)
g.addEdge(1, 2)
g.addEdge(2, 0)
g.addEdge(2, 3)
g.addEdge(3, 3)

print ("Following is Breadth First Traversal"
                  " (starting from vertex 2)")
g.BFS(2)
```

Python : BFS 2

The time complexity of the BFS algorithm is represented in the form of O(V + E), where V is the number of nodes and E is the number of edges.

The space complexity of the algorithm is O(V).

Ref : Time and space complexity https://thetapacademy.com/time-and-space-complexity-in-data-structure/

Time complexity is the amount of time taken to run an algorithm. It is the measure of the number of elementary operations performed by an algorithm and an estimate of the time required for that operation. It also depends on external factors such as the compiler, the processor's speed, etc.

If we ask you how much time you can add the first five natural numbers? (you start counting 1+2+3+4+5). Assume that, it took you 3 seconds. But how will you calculate this for the computer? We cannot! And thus, computer scientists have come up with an approach to calculate a rough estimate of the time taken to execute a code, called Time Complexity.

Space complexity is the amount of memory space an algorithm/program uses during its entire execution. It measures the number of variables created to store values, including both the inputs and the outputs. In simple terms, it is a rough estimation of how much storage your code will take in RAM.

```
import collections

   # BFS algorithm
   def bfs(graph, root):

   visited, queue = set(), collections.deque([root])
   visited.add(root)

   while queue:
         # Dequeue a vertex from queue
         vertex = queue.popleft()
         print(str(vertex) + " ", end="")
```

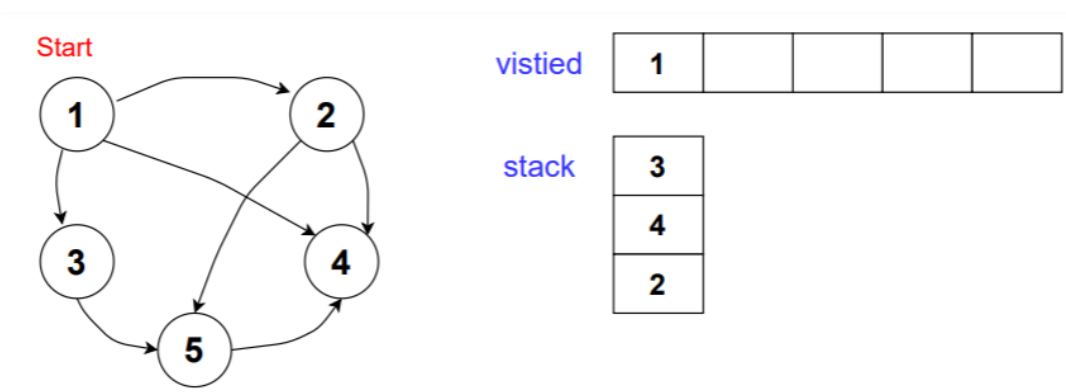
```
        # If not visited, mark it as visited, and
        # enqueue it
        for neighbour in graph[vertex]:
              if neighbour not in visited:
                    visited.add(neighbour)
                    queue.append(neighbour)


if __name__ == '__main__':
graph = {0: [1, 2], 1: [2], 2: [3], 3: [1, 2]}
print("Following is Breadth First Traversal: ")
bfs(graph, 0)
```

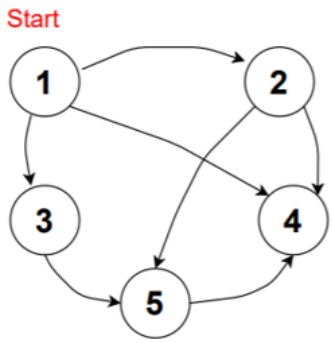
3.2 การท่องเข้าไปในแนวลึก (Depth First Traversal/Search: DFS)

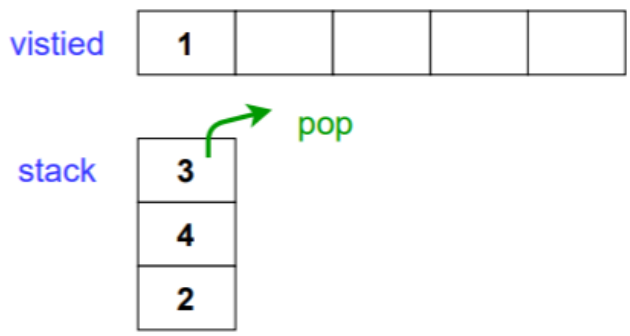
มีขั้นตอนการทำงานดังนี้

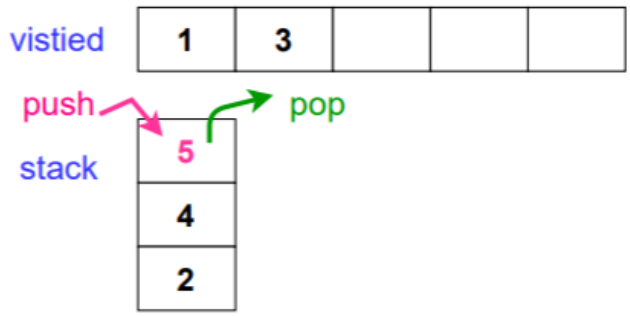
1. เลือกจุดในกราฟ ขึ้นมาเป็นจุดเริ่มต้น (visited)
2. นำจุดที่เชื่อมโยง/ประชิดกับจุดที่เลือก ไปเก็บไว้ในสแตก (push on Stack)
3. เอาข้อมูลออกจากสแตก (pop) และนำทุกจุดที่เชื่อมโยง/ประชิดกับจุดที่เอาออก (ถ้า มี) ไปเก็บไว้ในสแตก (push)
4. ทำซ้ำในขั้นตอนที่ 3 จนกระทั่งทุกจุดถูกแวะเยี่ยม

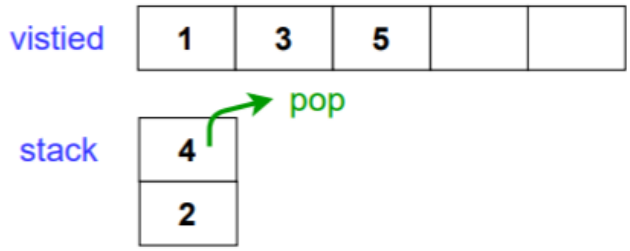
ตัวอย่าง DFS

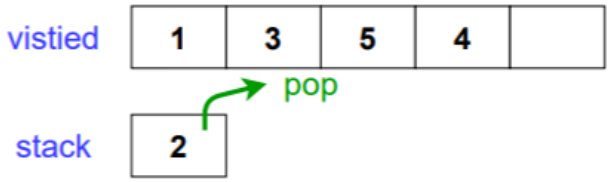


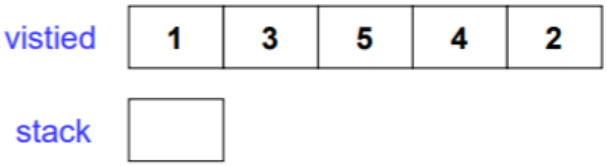

Start
1
2
3
4
5


vistied
1
pop
stack
3
4
2

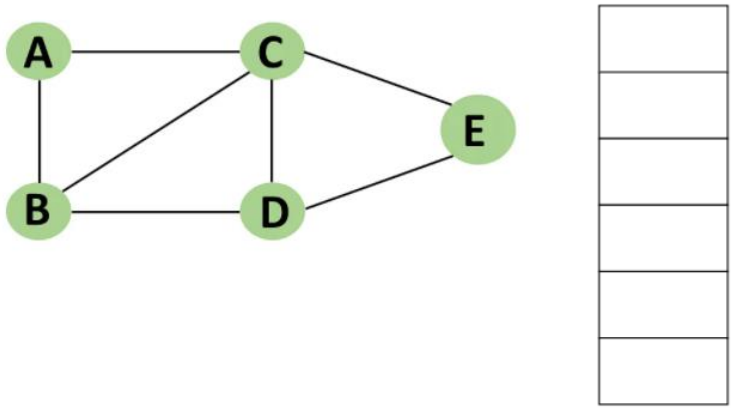

vistied
1
3
push
pop
stack
5
4
2


vistied
1
3
5
pop
stack
4
2

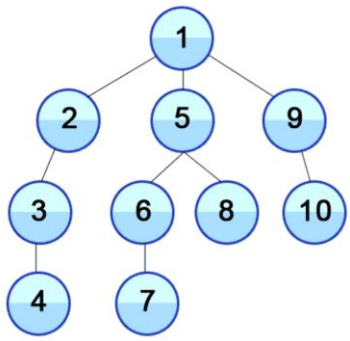

vistied
1
3
5
4
pop
stack
2


vistied
1
3
5
4
2
stack

<u>ตัวอย่าง</u> DFS

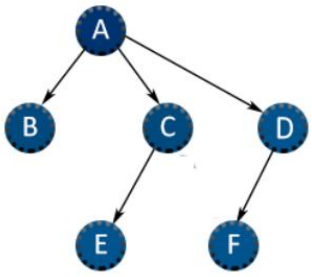


<u>ตัวอย่าง</u> DFS



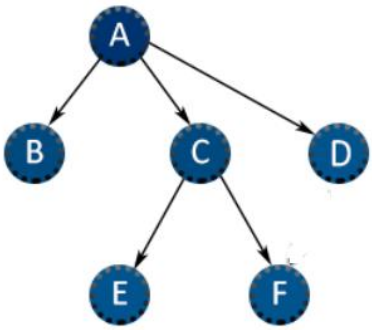
<u>ตัวอย่าง</u> เปรียบเทียบ BFS และ DFS

DFS



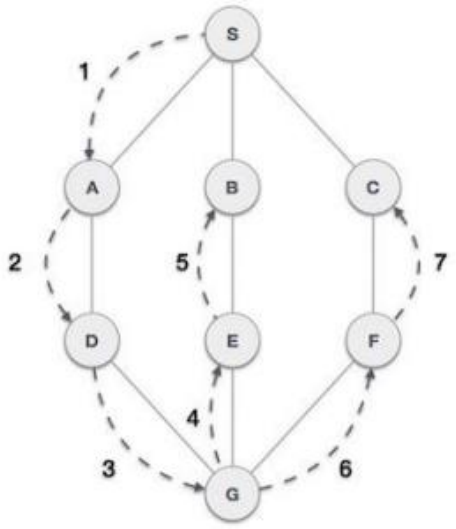
A D F C E B

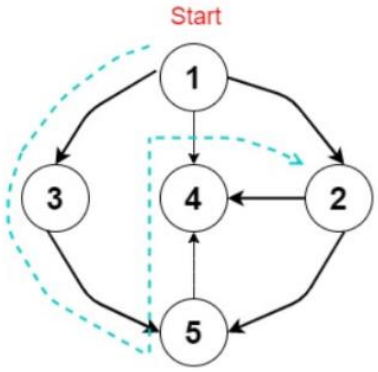
BFS

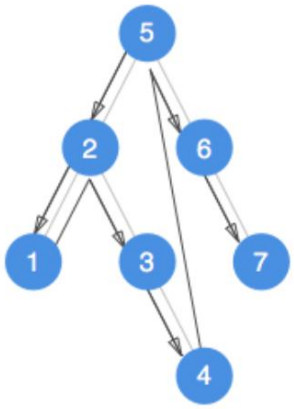


A B C D E F

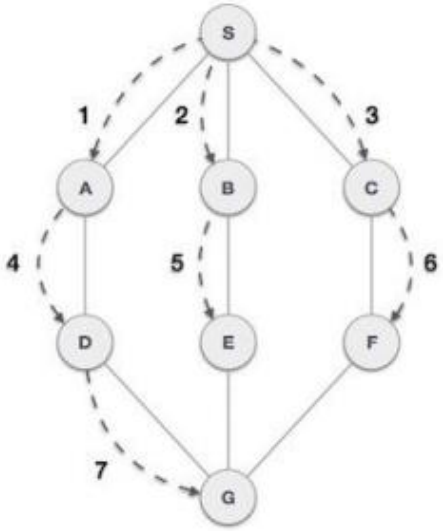
DFS

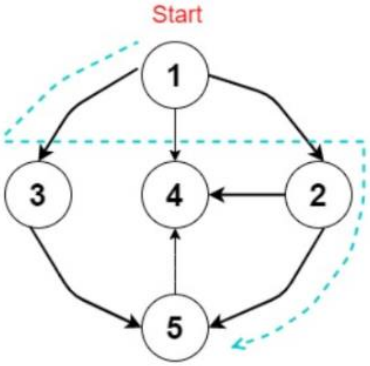

S
A
B
C
D
E
F
G
1
2
3
4
5
6
7

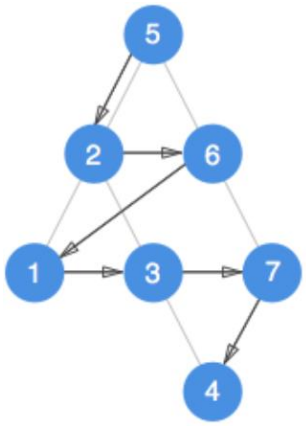

Start
1
3
4
2
5

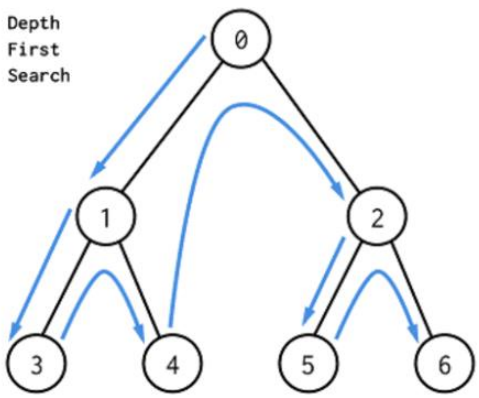

5
2
6
1
3
7
4

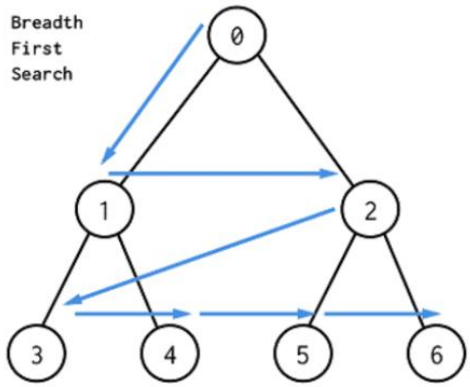
BFS


S
A
B
C
D
E
F
G
1
2
3
4
5
6
7

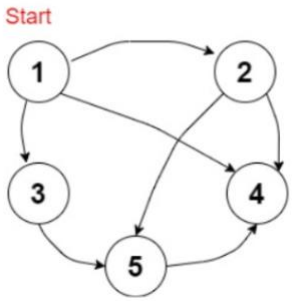

Start
1
3
4
2
5


5
2
6
1
3
7
4





คำถาม



1. BFS ลำดับการเลือกข้อมูลไปใส่ใน Queue มีผลกับผลลัพธ์ที่ได้หรือไม่
2. DFS ลำดับการเลือกข้อมูลไปใส่ใน Stack

<u>ตัวอย่าง</u> BFS

วิธีการเลือกแบบ 1

vistied	1				
queue	3	4	2		

วิธีการเลือกแบบ 2

vistied	1				
queue	2	4	3		

4. การหาระยะทางที่สั้นที่สุด (Shortest path) ด้วย Dijkstra's Algorithm

Dijkstra's algorithm ใช้ในการหาเส้นทางที่สั้นที่สุดจาก<u>จุดหนึ่ง</u>ในกราฟ ไปยัง<u>ทุกจุด</u>อื่น ๆ ในกราฟรูปเดียวกัน โดยที่ทุกจุดต้องมีเส้นทางเชื่อมโยงกลับไปยังจุดเริ่มต้นได้ ขั้นตอนวิธีของ Dijkstra เป็นดังนี้ (Wikipedia : shorturl.at/jyF37)

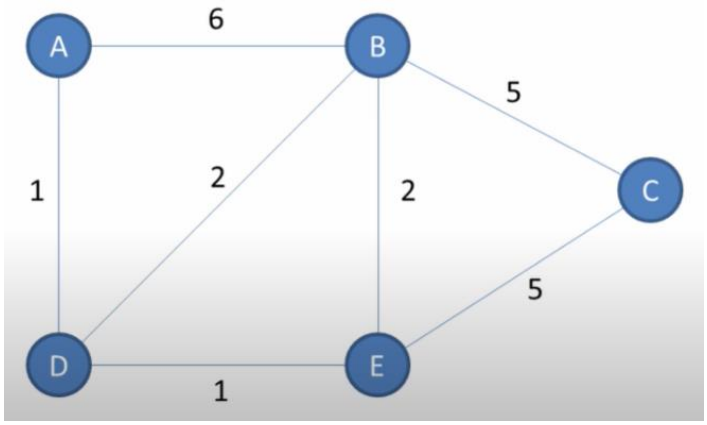
กำหนดให้จุดหนึ่งเป็นจุดเริ่มต้น (initial node) และกำหนดให้ "ระยะทางของจุด Y" (distance of node Y) หมายถึงระยะทางจากจุดเริ่มต้นไปยังจุด Y ขั้นตอนวิธีของไดก์สตราจะกำหนดค่าระยะทางเริ่มต้นไว้บางจุดและจะเพิ่มค่าไปทีละขั้นตอน

1. กำหนดให้ทุกจุดมีค่าระยะทางตามเส้นเชื่อม โดยให้จุดเริ่มต้นมีค่าเป็นศูนย์ และจุดอื่นมีค่าเป็นอนันต์
2. ทำเครื่องหมายทุกจุดยกเว้นจุดเริ่มต้นว่ายังไม่ไปเยือน (unvisited) ตั้งให้จุดเริ่มต้นเป็นจุดปัจจุบัน สร้างเซตของจุดที่ยังไม่ไปเยือนขึ้นมาเซตหนึ่งซึ่งประกอบด้วยทุกจุดยกเว้นจุดเริ่มต้น
3. จากจุดปัจจุบัน พิจารณาจุดข้างเคียงตามเส้นเชื่อมทุกจุดที่ยังไม่ไปเยือน และคำนวณระยะทางต่อเนื่องของเส้นเชื่อม ตัวอย่างเช่น ถ้าจุดปัจจุบันคือ A มีระยะทางของจุดเป็น 6 และเส้นเชื่อมที่ต่อจาก A ไปยังจุดข้างเคียง B มีระยะทางเป็น 2 ดังนั้นระยะทางของจุด B (โดยผ่าน A) จึงเท่ากับ 6+2=8 เป็นต้น ถ้าระยะทางที่คำนวณได้มีค่าน้อยกว่าค่าระยะทางที่บันทึกอยู่ของจุดนั้น ให้เขียนทับค่าระยะทางของจุดดังกล่าว แม้ว่าจุดข้างเคียงได้ถูกพิจารณาแล้ว แต่ก็ยังไม่ทำเครื่องหมายว่าไปเยือนแล้ว (visited) ในขั้นตอนนี้ จุดข้างเคียงจะยังคงอยู่ในเซตของจุดที่ยังไม่ไปเยือนเช่นเดิม
4. เมื่อพิจารณาจุดข้างเคียงจากจุดปัจจุบันครบทุกจุดแล้ว ทำเครื่องหมายจุดปัจจุบันว่าไปเยือนแล้ว และนำออกจากเซตของจุดที่ยังไม่ไปเยือน จุดที่ไปเยือนแล้วนี้จะไม่ถูกนำมาตรวจสอบอีก ค่าระยะทางที่บันทึกอยู่จะสิ้นสุดและมีค่าน้อยสุด
5. จุดปัจจุบันตัวถัดไปที่ถูกเลือกจะเป็นจุดที่มีค่าระยะทางน้อยสุดในเซตของจุดที่ยังไม่ไปเยือน
6. ถ้าเซตของจุดที่ยังไม่ไปเยือนว่างแล้วให้หยุดการทำงาน ขั้นตอนวิธีเสร็จสิ้น หากไม่ใช่ให้เลือกจุดยังไม่ไปเยือนที่มีค่าระยะทางน้อยสุดเป็นจุดปัจจุบัน แล้ววนกลับไปทำขั้นตอนที่ 3

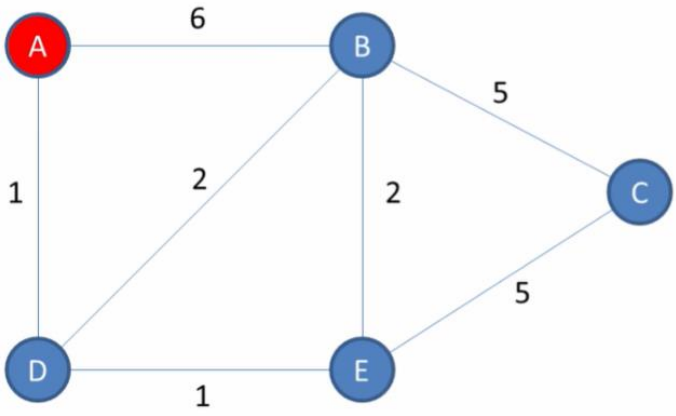
Dijkstra's Algorithm

```
Let distance of start vertex from start vertex = 0
Let distance of all other vertices from start = ∞ (infinity)

WHILE vertices remain unvisited
    Visit unvisited vertex with smallest known distance from start vertex (call this 'current vertex')
    FOR each unvisited neighbour of the current vertex
        Calculate the distance from start vertex
        If the calculated distance of this vertex is less than the known distance
            Update shortest distance to this vertex
            Update the previous vertex with the current vertex
        end if
    NEXT unvisited neighbour
    Add the current vertex o the list of visited vertices
END WHILE
```

<u>ตัวอย่าง</u> หา Shortest path จากจุด A ไปยังจุดต่าง ๆ ในกราฟ



Consider the start vertex, A
Distance to A from A = 0
Distances to all other vertices from A are unknown, therefore ∞ (infinity)

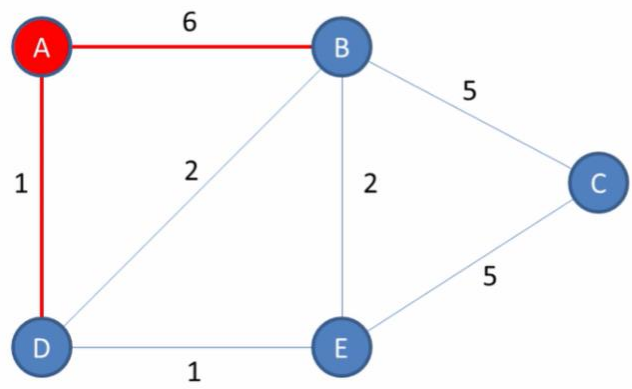


Vertex	Shortest distance from A	Previous vertex
A	0	
B	∞	
C	∞	
D	∞	
E	∞	

Visited = [ ] Unvisited = [A, B, C, D, E]

For the current vertex, examine its unvisited neighbours

We are currently visiting A and its unvisited neighbours are B and D

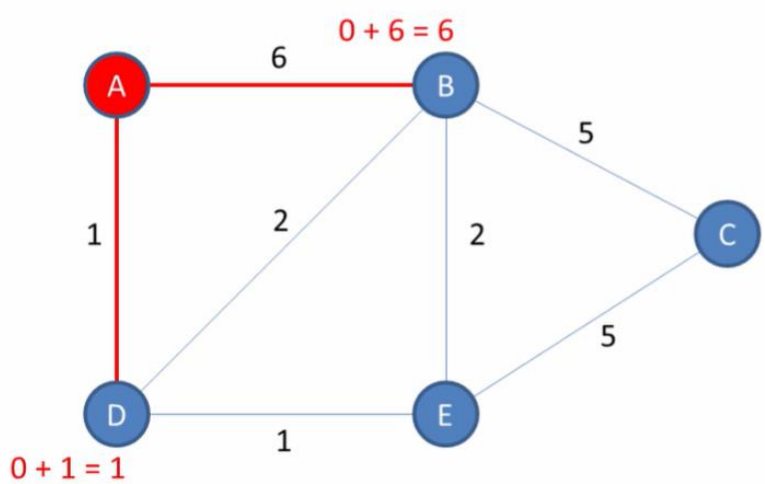


Vertex	Shortest distance from A	Previous vertex
A	0	
B	∞	
C	∞	
D	∞	
E	∞	

Visited = [ ] Unvisited = [A, B, C, D, E]

For the current vertex, calculate the distance of each neighbour from the start vertex

If the calculated distance of a vertex is less than the known distance, update the shortest distance

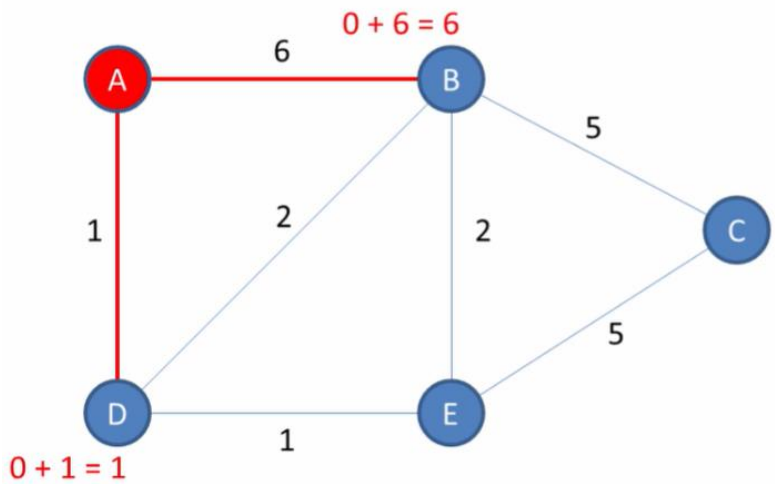


Vertex	Shortest distance from A	Previous vertex
A	0	
B	6	
C	∞	
D	1	
E	∞	

Visited = [ ] Unvisited = [A, B, C, D, E]

Update the previous vertex for each of the updated distances

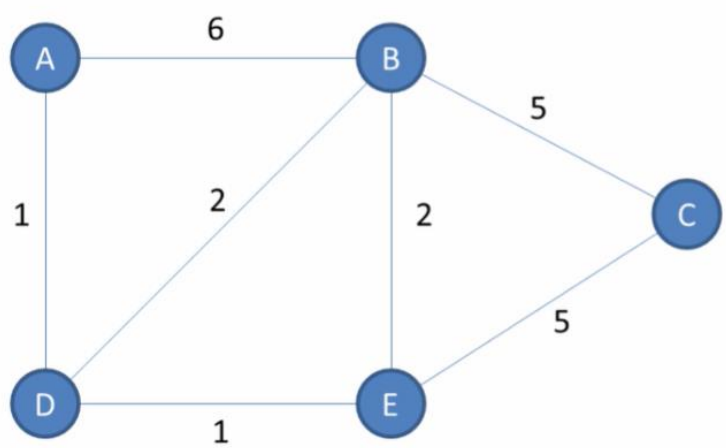
In this case we visited B and D via A



Vertex	Shortest distance from A	Previous vertex
A	0	
B	6	A
C	∞	
D	1	A
E	∞	

Visited = [ ] Unvisited = [A, B, C, D, E]

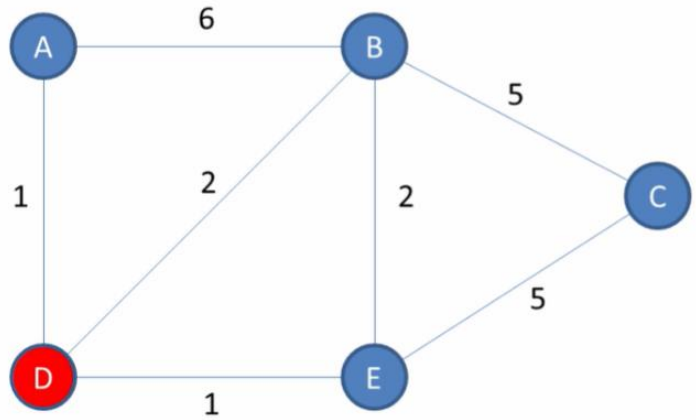
Add the current vertex to the list of visited vertices



Vertex	Shortest distance from A	Previous vertex
A	0	
B	6	A
C	∞	
D	1	A
E	∞	

Visited = [A] Unvisited = [B, C, D, E]

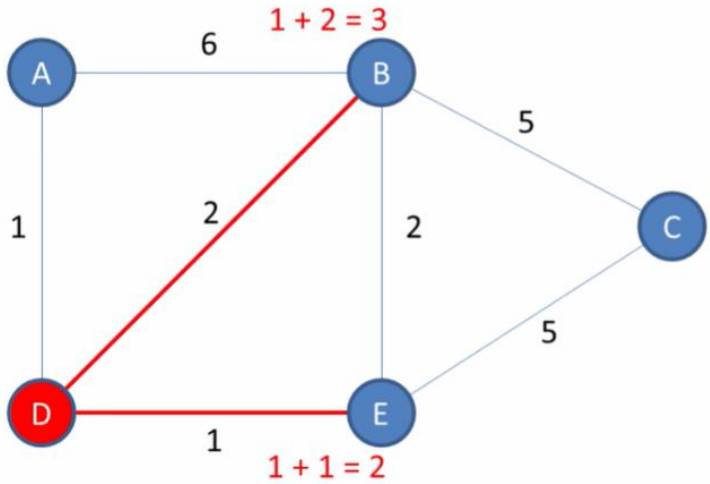
Visit the unvisited vertex with the smallest known distance from the start vertex
This time around, it is vertex D



Vertex	Shortest distance from A	Previous vertex
A	0	
B	6	A
C	∞	
D	1	A
E	∞	

Visited = [A] Unvisited = [B, C, D, E]

For the current vertex, calculate the distance of each neighbour from the start vertex

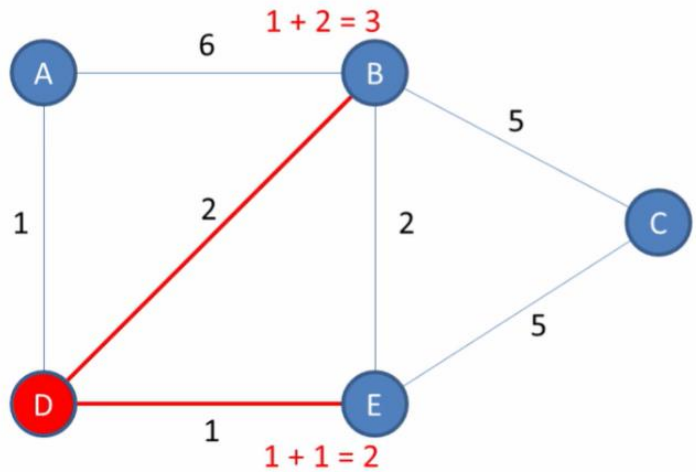


Vertex	Shortest distance from A	Previous vertex
A	0	
B	6	A
C	∞	
D	1	A
E	∞	

Visited = [A] Unvisited = [B, C, D, E]

Update the previous vertex for each of the updated distances

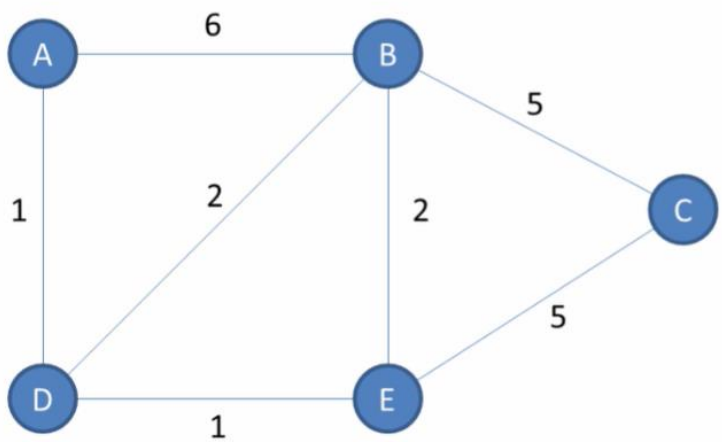
In this case we visited B and E via D



Vertex	Shortest distance from A	Previous vertex
A	0	
B	3	D
C	∞	
D	1	A
E	2	D

Visited = [A] Unvisited = [B, C, D, E]

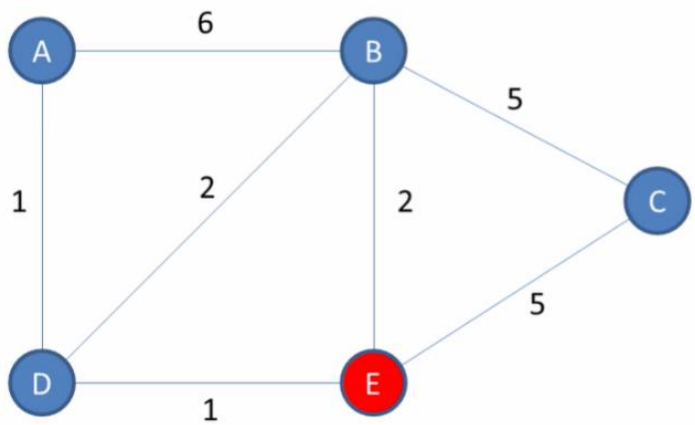
Add the current vertex to the list of visited vertices



Vertex	Shortest distance from A	Previous vertex
A	0	
B	3	D
C	∞	
D	1	A
E	2	D

Visited = [A, D] Unvisited = [B, C, E]

Visit the unvisited vertex with the smallest known distance from the start vertex
This time around, it is vertex E

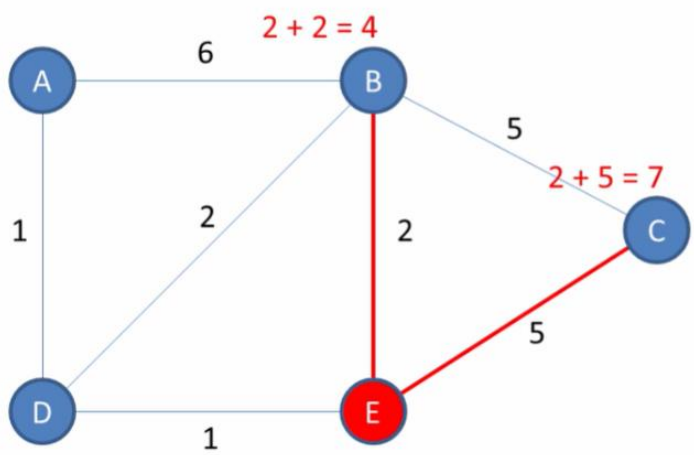


Vertex	Shortest distance from A	Previous vertex
A	0	
B	3	D
C	∞	
D	1	A
E	2	D

Visited = [A, D] Unvisited = [B, C, E]

For the current vertex, examine its unvisited neighbours
We are currently visiting E and its unvisited neighbours are B and C

For the current vertex, calculate the distance of each neighbour from the start vertex



Vertex	Shortest distance from A	Previous vertex
A	0	
B	3	D
C	∞	
D	1	A
E	2	D

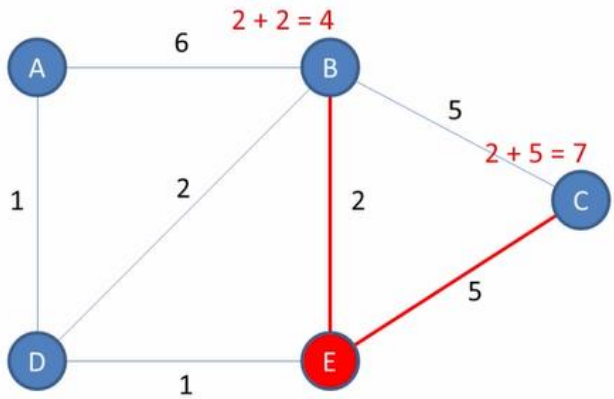
Visited = [A, D] Unvisited = [B, C, E]

If the calculated distance of a vertex is less than the known distance, update the shortest distance

We do not need to update the distance to B

Update the previous vertex for each of the updated distances

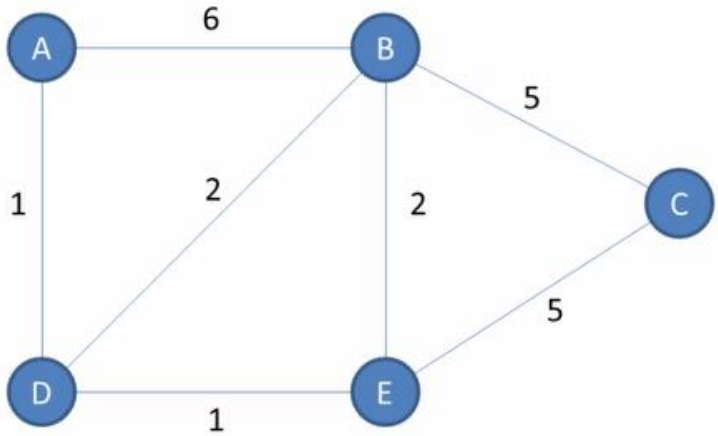
In this case we visited C via E



Vertex	Shortest distance from A	Previous vertex
A	0	
B	3	D
C	7	E
D	1	A
E	2	D

Visited = [A, D] Unvisited = [B, C, E]

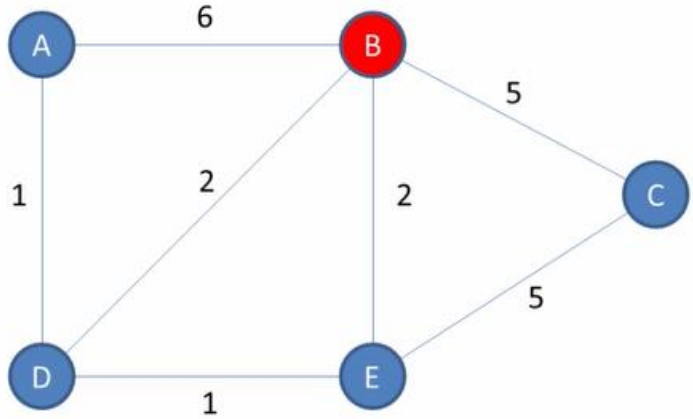
Add the current vertex to the list of visited vertices



Vertex	Shortest distance from A	Previous vertex
A	0	
B	3	D
C	7	E
D	1	A
E	2	D

Visited = [A, D, E] Unvisited = [B, C]

Visit the unvisited vertex with the smallest known distance from the start vertex
This time around, it is vertex B

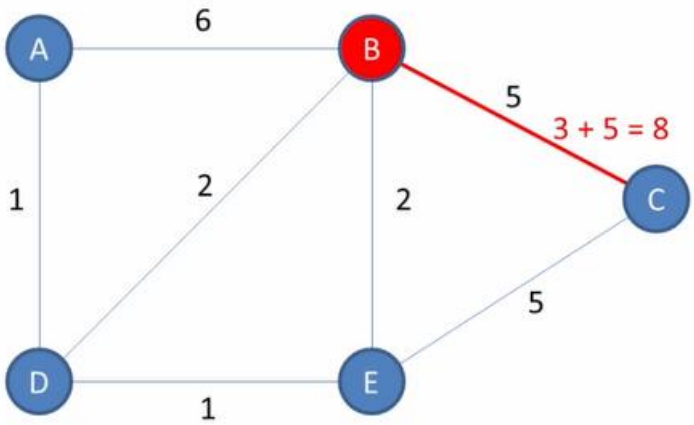


Vertex	Shortest distance from A	Previous vertex
A	0	
B	3	D
C	7	E
D	1	A
E	2	D

Visited = [A, D, E] Unvisited = [B, C]

For the current vertex, examine its unvisited neighbours
We are currently visiting B and its only unvisited neighbour is C

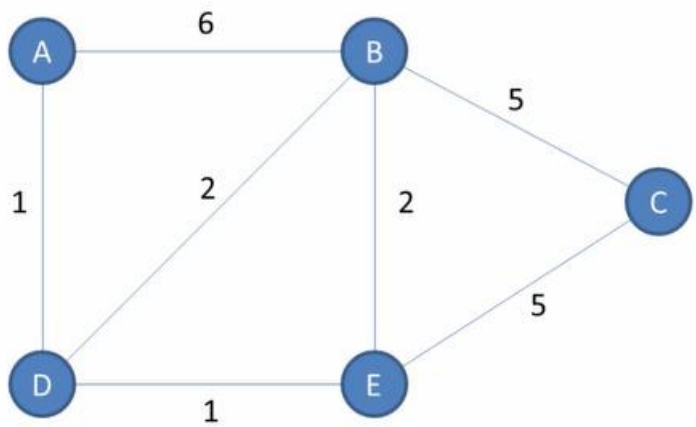
If the calculated distance of a vertex is less than the known distance, update the shortest distance
We do not need to update the distance to C



Vertex	Shortest distance from A	Previous vertex
A	0	
B	3	D
C	7	E
D	1	A
E	2	D

Visited = [A, D, E] Unvisited = [B, C]

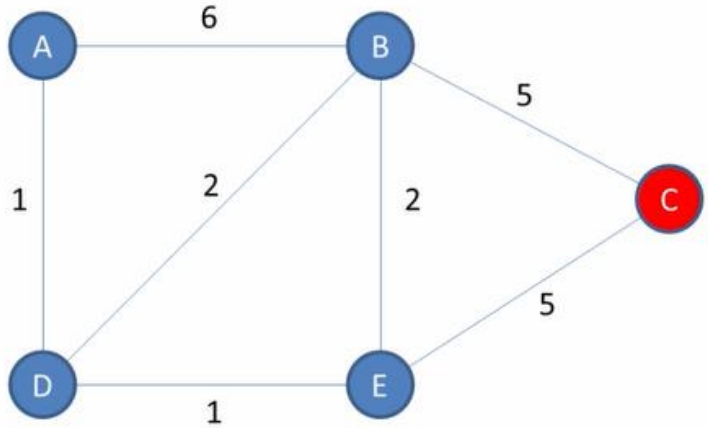
Add the current vertex to the list of visited vertices



Vertex	Shortest distance from A	Previous vertex
A	0	
B	3	D
C	7	E
D	1	A
E	2	D

Visited = [A, D, E, B] Unvisited = [C]

Visit the unvisited vertex with the smallest known distance from the start vertex
This time around, it is vertex C

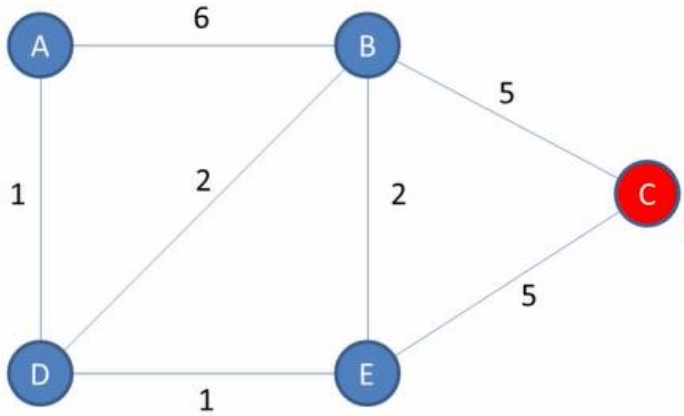


Vertex	Shortest distance from A	Previous vertex
A	0	
B	3	D
C	7	E
D	1	A
E	2	D

Visited = [A, D, E, B] Unvisited = [C]

For the current vertex, examine its unvisited neighbours

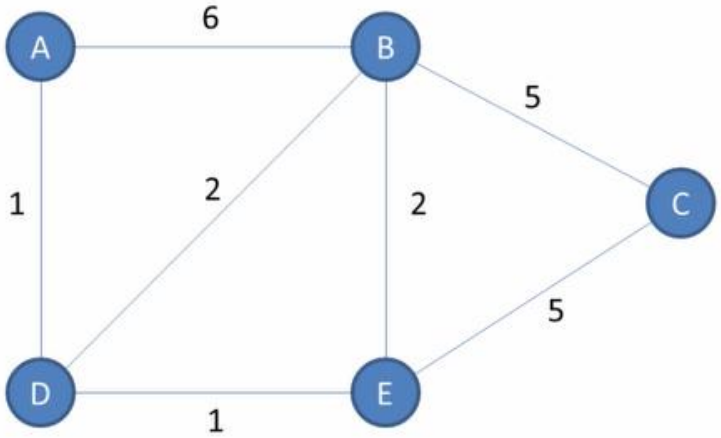
We are currently visiting C and it has no unvisited neighbours



Vertex	Shortest distance from A	Previous vertex
A	0	
B	3	D
C	7	E
D	1	A
E	2	D

Visited = [A, D, E, B] Unvisited = [C]

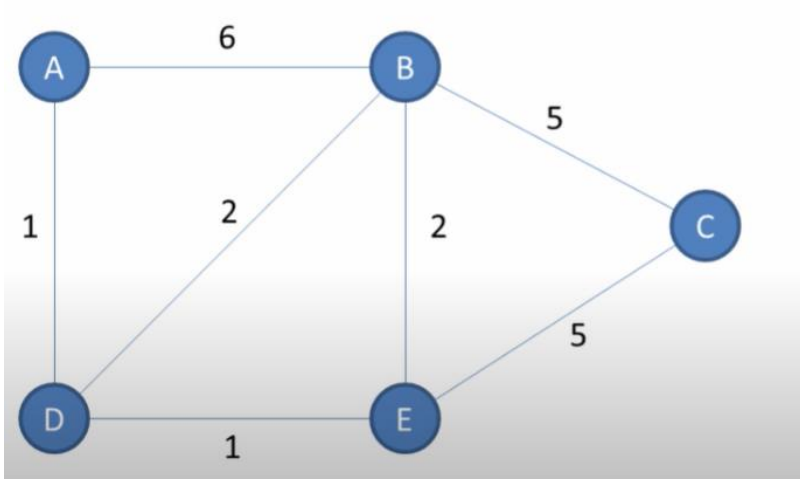
Add the current vertex to the list of visited vertices



Vertex	Shortest distance from A	Previous vertex
A	0	
B	3	D
C	7	E
D	1	A
E	2	D

Visited = [A, D, E, B, C] Unvisited = []

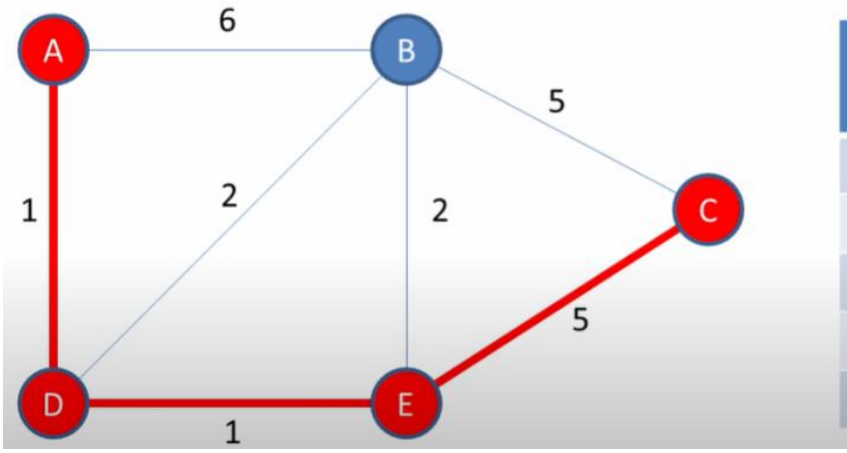
ผลลัพธ์



Vertex	Shortest distance from A	Previous vertex
A	0	
B	3	D
C	7	E
D	1	A
E	2	D

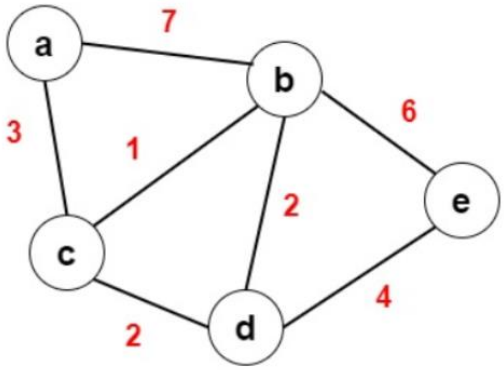
ความหมายของช่อง Previous vertex แสดงดังภาพข้างล่าง

We arrived at D via A Path = A → D → E → C



Vertex	Shortest distance from A	Previous vertex
A	0	
B	3	D
C	7	E
D	1	A
E	2	D

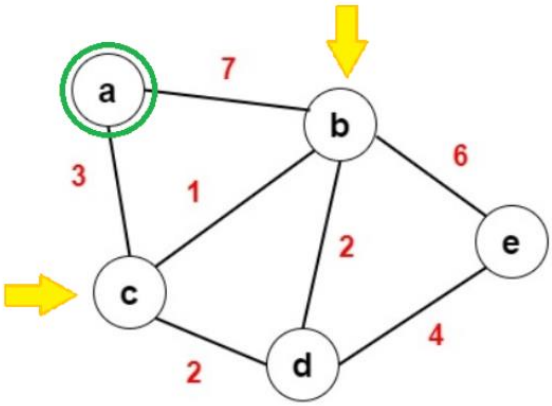
<u>ตัวอย่าง</u> จงหาระยะที่สั้นที่สุดจากจุด a ไปยังทุกจุดอื่นๆในกราฟ



จุด	ระยะทางสั้นที่สุด จากจุด a	จุดก่อนหน้า
a	0	
b	∞	
c	∞	
d	∞	
e	∞	

visited = {

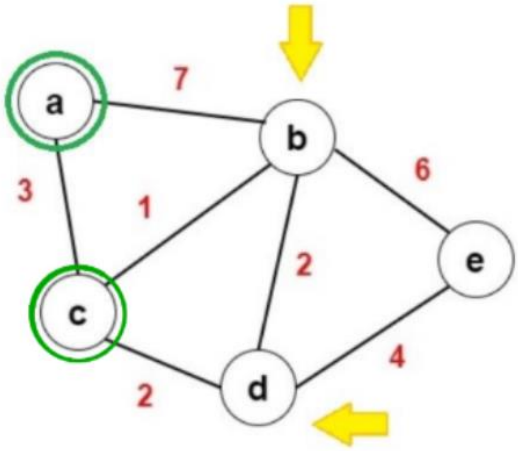
unvisited = {a, b, c, d, e}



จุด	ระยะทางสั้นที่สุด จากจุด a	จุดก่อนหน้า
a	0	-
b	~~∞~~ 7	a
c	~~∞~~ 3	a
d	∞	
e	∞	

visited = { a

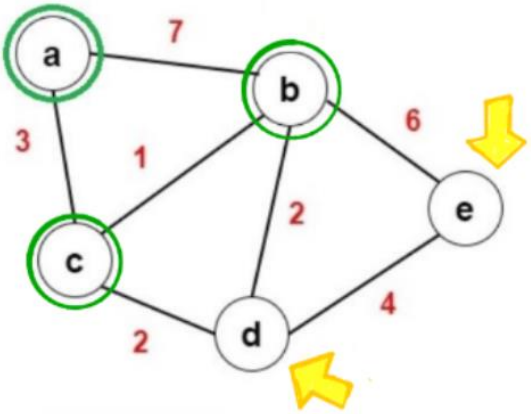
unvisited = {~~a~~, b, c, d, e}



visited = { a, c

unvisited = {~~a~~, b, ~~c~~, d, e}

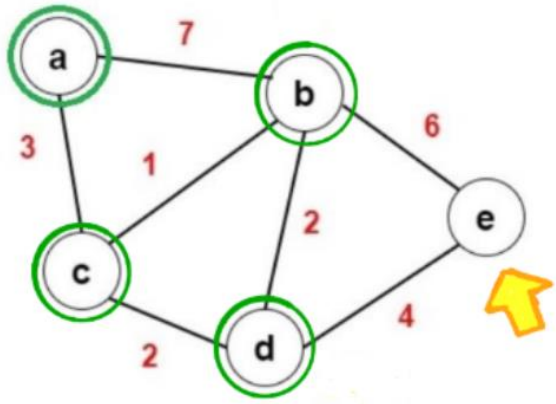
จุด	ระยะทางสั้นที่สุด จากจุด a	จุดก่อนหน้า
a	0	-
b	~~∞~~ ~~7~~ 4	~~a~~ c
c	~~∞~~ 3	a
d	~~∞~~ 5	c
e	∞	



visited = { a, c , b

unvisited = {~~a~~, ~~b~~, ~~c~~, d, e}

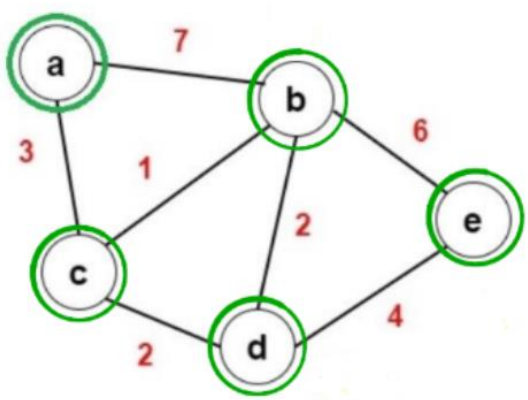
จุด	ระยะทางสั้นที่สุด จากจุด a	จุดก่อนหน้า
a	0	-
b	~~∞~~ ~~7~~ 4	~~a~~ c
c	~~∞~~ 3	a
d	~~∞~~ 5	c
e	~~∞~~ 10	b



จุด	ระยะทางสั้นที่สุด จากจุด a	จุดก่อนหน้า
a	0	-
b	~~∞~~ ~~7~~ 4	~~a~~ c
c	~~∞~~ 3	a
d	~~∞~~ 5	c
e	~~∞~~ ~~10~~ 9	~~b~~ d

visited = {a, c, b, d

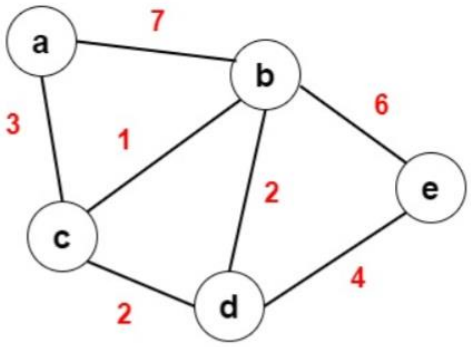
unvisited = {~~a~~, ~~b~~, ~~c~~, ~~d~~, e}



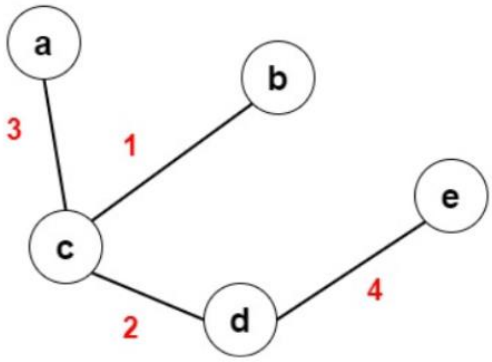
จุด	ระยะทางสั้นที่สุด จากจุด a	จุดก่อนหน้า
a	0	-
b	~~∞~~ ~~7~~ 4	~~a~~ c
c	~~∞~~ 3	a
d	~~∞~~ 5	c
e	~~∞~~ ~~10~~ 9	~~b~~ d

visited = {a, c, b, d, e}

unvisited = {~~a~~, ~~b~~, ~~c~~, ~~d~~, ~~e~~}

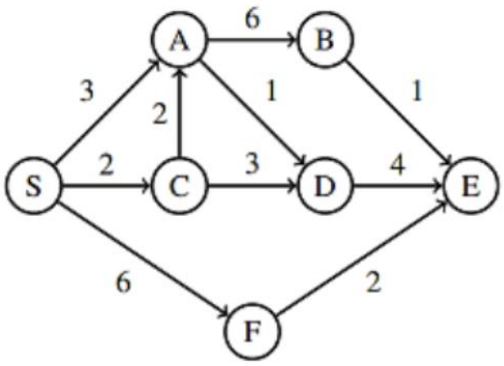


กราฟตั้งต้น



กราฟที่มีทางเดินสั้นที่สุดจากจุด a

<u>ตัวอย่าง</u> จงหาระยะที่สั้นที่สุดจากจุด S ไปยังทุกจุดอื่นๆในกราฟ



Visited = {

Unvisited = { S, A, B, C, D, E, F }

จุด	ระยะทางสั้นที่สุดจากจุด S	จุดก่อนหน้า
S	0	
A		
B		
C		
D		
E		
F		

<u>ตัวอย่าง</u> YouTube

1. https://www.youtube.com/watch?v=pVfj6mxhdMw
2. https://www.youtube.com/watch?v=wtdtkJgcYUM